

Projeto Prático de POO: J-SoLong (Jogo 2D Grid-Based)

O objetivo é desenvolver um jogo em que um personagem precisa coletar todos os itens espalhados por um mapa, escapar por uma saída e respeitar regras de colisão e parsing.

1. Objetivos Pedagógicos de POO

- **Análise e Validação de Arquivos (Parsing):** Ler e validar um mapa em disco (*.ber), tratando exceções de E/S e formatação sintática incorreta.
- **Polimorfismo e Hierarquia de Classes:** Tratar os elementos do mapa (Paredes, Espaços Vazios, Coletáveis, Jogador, Saída, Inimigos) como objetos derivados de uma classe base comum (ElementoMapa ou Entidade).
- **Encapsulamento e Ocultamento de Informação:** Proteger o estado do jogo e as matrizes/coleções internas contra modificações diretas, expondo apenas comportamentos seguros.
- **Padrão MVC (Model-View-Controller) / Separação de Camadas:** Isolar a lógica das regras do jogo (Backend) do motor de renderização gráfica e captura de eventos da GUI (Frontend).

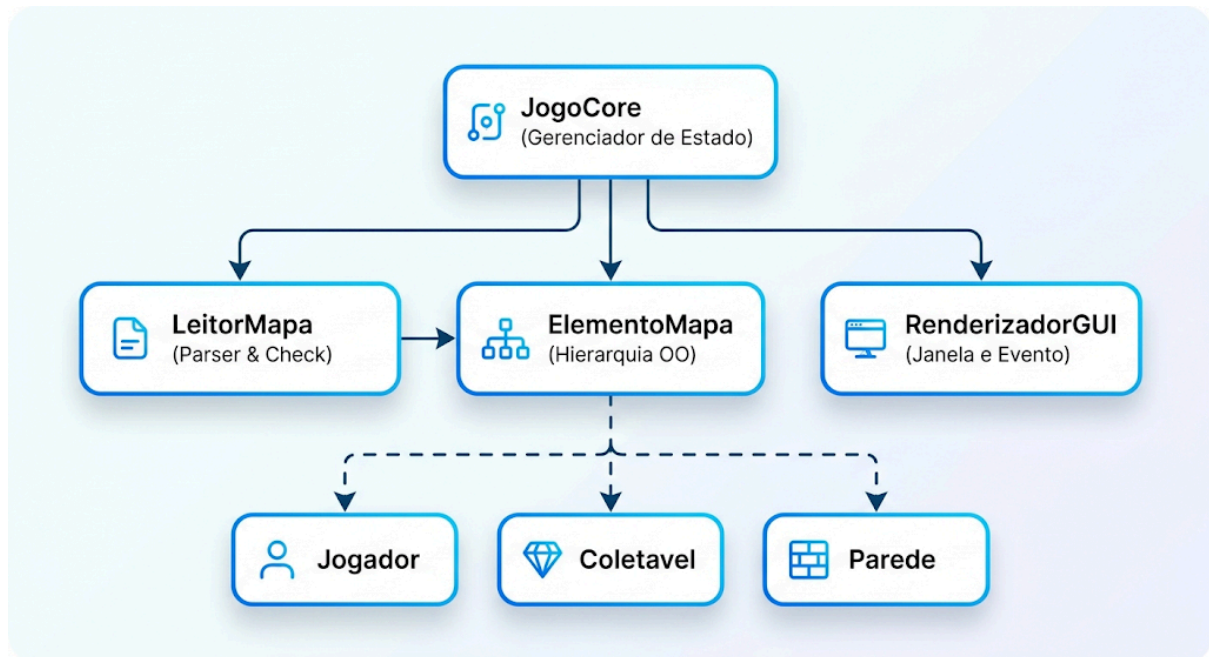
2. Requisitos Funcionais do Jogo

- **Carregamento e Validação do Mapa (.ber):**
 - O mapa deve ser retangular e inteiramente cercado por paredes (1).
 - Deve conter exatamente 1 Posição Inicial (P), 1 Saída (E) e pelo menos 1 Coletável (C).
 - O programa deve validar se existe um **caminho válido** (Valid Path) para que o jogador consiga pegar todos os coletáveis e alcançar a saída (algoritmo como *Flood Fill*).
 - Caso o arquivo apresente irregularidades, o sistema deve tratar a exceção e exibir uma mensagem de erro clara sem encerrar abruptamente de maneira inconsistente.
- **Mecânica de Jogo:**
 - Movimentação ortogonal do jogador via teclado (Cima, Baixo, Esquerda, Direita).
 - Impedimento rigoroso de colisão contra paredes (1).
 - Contagem e exibição do número de movimentos no terminal ou na própria interface.
 - A vitória só ocorre ao tocar na Saída (E) após coletar **todos** os itens (C).

- **Gerenciamento Gráfico:**

- Renderização de *sprites* 2D / *tiles* para cada célula do mapa em uma janela gráfica.
- Fechamento limpo da janela e liberação de recursos ao pressionar **ESC** ou clicar no botão fechar.

3. Modelo de Arquitetura Orientada a Objetos



Principais Classes de Domínio:

- **ElementoMapa** (*Classe Abstrata*): Define posição (x, y) , caminho do *sprite* e comportamento base de colisão.
- **Jogador**, **Coletavel**, **Parede**, **Saida** (*Subclasses*): Implementam comportamentos específicos (ex.: **Coletavel.aoColetar()**, **Saida.tentarEscapar()**).
- **Mapa**: Encapsula a matriz de elementos, realizando a verificação de limites e movimentação.
- **LeitorMapa**: Classe utilitária responsável pelo parsing do arquivo **.ber**, lançamento de exceções personalizadas de mapa inválido e validação de caminho navegável.

4. Estrutura do Projeto (1 Alunos)

- **Motor do Jogo, Parser e Regras de Domínio (Model)**

- Implementação da leitura e validação dos arquivos **.ber** (validação de bordas, contagem de caracteres e Flood Fill para verificação de caminho).
- Criação da hierarquia de classes (**ElementoMapa**, **Jogador**, **Coletavel**, **Mapa**) e tratamento de exceções de jogo.
- Controle de regras de vitória, contagem de movimentos e gerenciamento de inventário/coletáveis.
- **Interface Gráfica, Sprites e Eventos de Entrada (View / Controller)**
 - Mapeamento dos objetos do modelo para a renderização gráfica em janela por meio da biblioteca da linguagem.
 - Captura de eventos de teclado para disparar as chamadas de movimentação no modelo.
 - Gestão de recursos visuais (carregamento de imagens/sprites) e encerramento seguro da aplicação (**ESC** / Fechar Janela).

5. Recursos Bônus (Opcionais)

- **Patrulha de Inimigos:** Adicionar uma classe de inimigos que se movimentam pelo mapa e fazem o jogador perder o jogo ao colidir.
- **Animação de Sprites:** Implementar troca de quadros (*frames*) para animar o personagem ou os coletáveis em tempo de execução.
- **Placar na Tela:** Exibir o contador de passos e quantidade de coletáveis restantes diretamente desenhados na interface visual, em vez de imprimir apenas no terminal.

6. Entregas Esperadas

- **Código-Fonte:** Projeto estruturado segundo os princípios de POO, separado em pacotes/módulos de domínio e visão.
- **Conjunto de Mapas de Teste (.ber):** Incluir mapas válidos e uma coleção de mapas inválidos (sem saída, sem paredes externas, sem caminho possível) para testes da etapa de parsing.
- **Documentação (README.md):** Arquivo contendo a descrição do projeto, diagrama de classes UML simples, instruções de compilação/execução e o relato sobre o uso de ferramentas de auxílio durante o desenvolvimento.
- **Apresentação do Jogo:** Demonstração da aplicação carregando um mapa, bloqueando movimentos inválidos, atualizando a janela e encerrando adequadamente.